

CareerConnect — Smart Campus Placement Management System

Improved Project Proposal

1. Problem Statement

(unchanged from original — already strong)

The campus recruitment process in many colleges is still managed through spreadsheets, emails, messaging applications, and manual coordination. This creates difficulties for students, recruiters, and placement officers in managing job opportunities, eligibility criteria, applications, shortlisting, interviews, and placement records.

Recruiters may receive hundreds of applications for a single job and have to manually check each candidate's CGPA, branch, skills, projects, certifications, internships, and other qualifications — a time-consuming and error-prone process.

2. Proposed Solution

CareerConnect is a full-stack web-based campus placement management platform that centralizes the complete recruitment process for students, recruiters, and placement administrators — with a **recruiter-configurable, explainable candidate ranking engine** at its core.

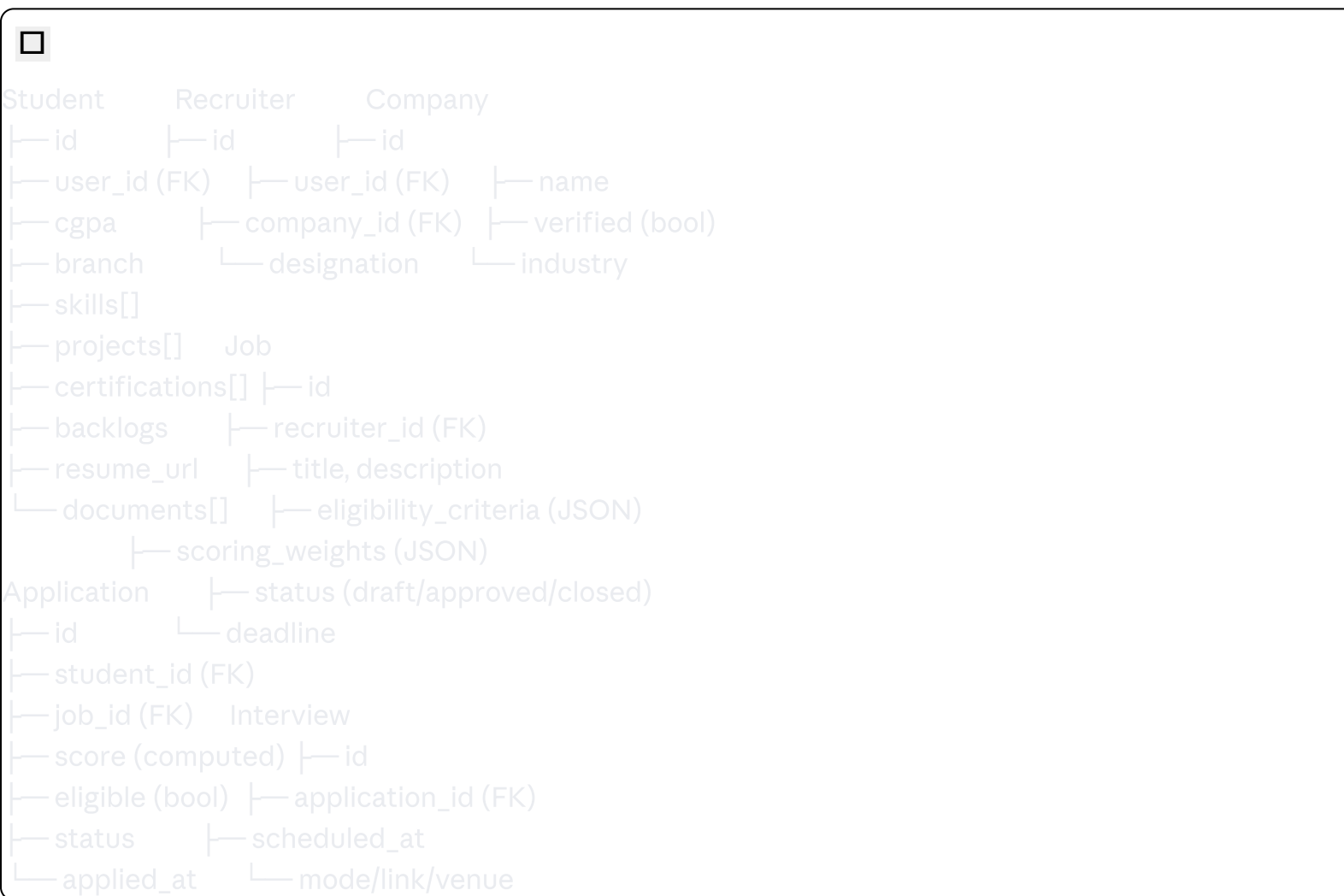
3. Tech Stack (New)

Layer	Technology	Why
Frontend	React (Next.js) + TailwindCSS	SSR for SEO on job listings, fast dev, component reuse across 3 dashboards
Backend	Node.js + Express (or NestJS for structure)	Async I/O suits notification/email-heavy workload
Database	PostgreSQL	Relational data (students, jobs, applications) with strong constraint support for eligibility rules
Cache/Queue	Redis + BullMQ	Background jobs: scoring recalculation, email/notification dispatch

Layer	Technology	Why
File Storage	AWS S3 / Supabase Storage (encrypted)	Resumes, certificates, document vault
Auth	JWT + refresh tokens, bcrypt/argon2 for passwords	Stateless auth, role-based access
Search	PostgreSQL full-text search (or Elasticsearch if scale demands)	Job search/filter
Hosting	Vercel (frontend) + Render/Railway/AWS EC2 (backend)	Simple, scalable for a campus-scale user base

(Any equivalent stack — Django/DRF, Spring Boot, MERN — works equally well; the point is to commit to one and justify it.)

4. Data Model (New — core entities)



`eligibility_criteria` and `scoring_weights` are stored as JSON so recruiters can define arbitrary rules per job without schema changes — this is what makes the "custom criteria" feature actually buildable.

5. Eligibility & Scoring Engine (New — the core differentiator, now specified)

This was the most interesting idea in your proposal but the least specified. Here's a concrete design:

Step 1 — Eligibility (hard filter, pass/fail)

Recruiter sets mandatory thresholds, e.g.:

☐

```
json
{
  "min_cgpa": 7.0,
  "allowed_branches": ["CSE", "IT", "ECE"],
  "max_backlogs": 0,
  "required_skills": ["Python", "SQL"]
}
```

A student who fails **any** condition is marked `ineligible` and excluded from ranking — no scoring is wasted on them.

Step 2 — Scoring (soft ranking, 0-100)

Recruiter assigns weights (must sum to 100) to scoring factors:

Factor	Example Weight	How it's computed
CGPA	30%	<code>(cgpa / 10) × 30</code>
Skill match	30%	<code>(matched_skills / required_skills) × 30</code>
Projects	15%	<code>min(project_count, 5)/5 × 15</code>
Certifications	10%	<code>min(cert_count, 3)/3 × 10</code>
Internship experience	15%	<code>has_internship ? 15 : 0</code> (or scaled by months)

Worked example: Student — CGPA 8.4, matches 3/4 required skills, 4 projects, 2 certifications, has an internship:

☐

CGPA:	$(8.4/10) \times 30$	= 25.2
Skills:	$(3/4) \times 30$	= 22.5
Projects:	$(4/5) \times 15$	= 12.0
Certs:	$(2/3) \times 10$	= 6.7
Internship:	15	

Total Score: 81.4 / 100		

This score ranks eligible candidates. Recruiters see a **ranked, sortable list with a breakdown** of how each score was computed (transparency matters — a black-box number invites distrust). Final shortlist/reject decision stays with the recruiter, as your original proposal correctly insists.

(This is a rules-based weighted-sum model — not ML. That's a deliberate, defensible choice: it's explainable, doesn't need training data, and is exactly the kind of "automation with human final say" your objectives call for. It can be extended to ML-based ranking later as a v2 feature.)

6. Job Recommendation Engine (New)

Reuses the scoring logic in reverse: for a given student profile, run it against all *open jobs*' criteria and surface the top-N by predicted score — "Jobs you're likely eligible for and would score well on," rather than a generic feed.

7. Security & Privacy Design (New)

RBAC: Student / Recruiter / Admin roles enforced at the API middleware level, not just UI hiding.

Document vault: files stored in a private S3 bucket/prefix per student, access via short-lived signed URLs only — never public links.

Data at rest: encrypted storage for CGPA, personal documents, contact info.

Data in transit: HTTPS everywhere.

Recruiter visibility limits: recruiters see only applicant data relevant to their job posting, not the full student database.

Audit log: track who viewed/downloaded a student's documents (important for trust in a placement system).

8. Notifications (New — specify the mechanism)

In-app notification center (real-time via WebSocket or polling)

Email notifications (application status change, interview scheduled, deadline reminders) via a queued job (BullMQ/Redis) so bulk sends don't block requests

9. Admin Analytics Dashboard (New — concrete metrics, not a placeholder)

Overall placement percentage, branch-wise placement percentage
Average / highest / median package (CTC)
Applications-per-job and funnel drop-off (applied → eligible → shortlisted → interviewed → hired)
Company-wise and recruiter-wise activity
Time-to-hire per drive

10. Non-Functional Requirements (New)

Should comfortably support a few thousand students and hundreds of concurrent job postings (typical campus scale) without needing distributed infra on day one
Scoring recalculation for a job with hundreds of applicants should run as a background job, not block the request
99%+ uptime during placement season (peak load), with monitoring/alerts

11. Objectives, Users, Expected Outcome

(unchanged — these sections were already clear and well-written in your original draft)

Summary of What Changed

Gap identified	Fix added
Scoring engine was a black box	Concrete weighted formula + worked example
No tech stack	Full stack table with justification
No data model	Core entity/schema sketch
"Secure" was asserted, not designed	RBAC, encryption, signed URLs, audit log
Recommendations undefined	Reuses scoring engine against open jobs
Analytics was a placeholder	Named metrics recruiters/admins actually need
No notification design	Queue-based email + in-app center
No scale considerations	NFR section added